GoMesh · system architecture

End-to-end encrypted messenger. Go relay server, Go client engine, SwiftUI app for macOS and iOS, binary protocol over TLS 1.3.

Mehul Raj · github.com/mehul-raj5/GoMesh

ciphertext · control / metadata · acknowledgement · n flow, see key below

ALICE'S DEVICE

plaintext and private keys stay here

SwiftUI app
macOS 14+ · iOS 17+ · 22 themes

Terminal client
same engine, command line

bridge (gomobile)
XCFramework · JSON calls + events

core · client engine (Go)

Client · connection
TLS dial, certificate pin check
reconnect with backoff, token resume
acker: every 64 packets or 250 ms

IdentityManager
X25519 identity · Ed25519 signing
signed prekeys, retired kept 30 days
login proof · safety numbers

SessionManager · ratchet
3× X25519 handshake → HKDF
two direction roots, rotate every 50
skipped keys ≤ 1000, kept 7 days

Groups · sender keys
verifies signed, hash-chained log
sender key: new every 100 msgs / 7 d
Ed25519-signed group messages

StateStore
sealed records, HMAC file names
state saved before ciphertext leaves
atomic write, then rename

History
history.db (SQLite)
each row sealed per account key
kept after sign-out, read-only

Media
XChaCha20-Poly1305 key per file
SHA-256 check, JPEG thumbnail
upload via signed URL

Receipts
clock → ✓ accepted → ✓✓ delivered
read receipts encrypted end to end
redelivered copies ignored

Vault · Keychain (Secure Enclave-wrapped on Apple)
account key: the root every sealed file and record is encrypted under
identity keys sealed under it · session token (32 B)
sign-out deletes the account key first, so nothing left on disk opens

1 2

TLS 1.3 · server certificate pinned by SHA-256 fingerprint · 55-byte header + body

RELAY SERVER · server/ · one Go binary

sees headers and timing, never plaintext

Listener
TLS 1.3 · :8080

Connection
30 s login deadline
1 KiB pre-auth cap

Authenticate
register · login
resume with token

Throttle + hash
per-IP window
argon2id slots

Key proof
32 B nonce →
64 B Ed25519 sig

LoginAck · MyGroups · drain queue

readLoop
stamp SenderID, drop if retired

handlePacket
dispatch 14 client types

Connection registry
one live device per account

Accounts & devices 5
register: unique name, argon2id hash
replaceDevice: revoke token, drop queue
broadcastKeyChange to every contact
session tokens stored as SHA-256

Group log
verify Ed25519 signature + hash chain
member keys checked against directory
order versions, fan out with bundles
ensurePairwiseConvs for sender keys

Routing & directory
handleData: sender must be a member
1:1-only types never fanned to groups
DirectInit: conv ID = SHA-256(IDs)
roster lookups answered to members only

Receipts & media
RecordAccepted → CtrlMsgAccepted
MarkDelivered → CtrlMsgDelivered
kept 14 days, replayed on request
signs Supabase upload/download URLs

DELIVERY

sendPacket
per recipient
online or not?

online → **send chan**
buffer of 128

writeLoop
≤ 1024 unacked
60 s ack timeout

3

offline → **queueOffline**
SQLite queue · 5000 packets / 64 MiB, oldest out

acknowledge
CtrlDelivered → Dequeue

stored until acked

On disconnect, unacknowledged packets return to the queue:
delivery is at least once and clients drop duplicates.

SQLITE · server.db

WAL · synchronous NORMAL

users	username · argon2id hash · 164 B bundle
devices	token SHA-256 · revoked: logout / replaced
conversations	1:1 and groups, with names
conv_members	who is in each 1:1 chat
group_log	version → signed change
queue	ciphertext per device, until acked
receipts	accepted / delivered, 14 days

Supabase Storage
encrypted blobs only, path <conv hex>/<random>
server signs short-lived upload and download URLs

OTHER DEVICES

Bob · online
write loop delivers at once
rebuilds the session from the first header
replies CtrlDelivered, then a read receipt

Carol · offline
packets wait in the SQLite queue
drained in order at her next login
group message: same bytes as Bob's

Alice's old Mac 5
after Alice signs in on a new device:
gets DeviceReplaced and wipes its keys
old queue deleted · contacts get KeyChange
groups re-keyed by a signed rekey change

4 file bytes are encrypted on the device and uploaded straight to storage; the relay only signs the URL

KEYS AND HOW LONG THEY LIVE

account key	Keychain	until sign-out or device move
identity · signing	sealed on device	same; public halves on server
signed prekey	sealed on device	rotation not yet scheduled
session token	Keychain	until logout; server keeps SHA-256
message key	memory	one message, then wiped
ratchet key	session state	until peer moves past it
group sender key	group state	100 messages / 7 days / a member leaves
file key	inside the message	one file

PACKET ANATOMY · bytes, drawn to scale

Every packet: header (IDs 16 + 16 + 16, type 1, flags 2, length 4)

header 55 — 55 B

First private message, "hiii"

header 55 | ratchet 50 | handshake 68 | 24 | 16 — 217 B

Group message, "hello all": encrypted once, same bytes to everyone

header 55 | group hdr 46 | signature 64 | 24 | 16 — 214 B

FLOWS

1. Sign in: password + 64 B proof over a 32 B challenge
2. Private message: ratcheted, rebuilt from its header
3. Group: sender keys over 1:1 sessions, one signed ciphertext
4. File: encrypted blob direct to storage
5. Device move: old device wiped, contacts and groups re-keyed

SERVER NEVER SEES

plaintext · private keys · sender keys · file contents

Unaudited learning project. Directory trusted until safety numbers match.